You

Let's say I have a bunch of past Fifa card types (as images) stored in a folder on GitHub. How can I make a web app using html, css and vanilla Js and hosted on GitHub pages that is a "Guess the card type" quiz. The names of the card type image files are the Card type that should be guessed. The web app should display the card type and allow an input box for the player to guess the card. The card type should appear at random (so any card type from the folder can be next) and then after the user clicks the "Submit Guess" button then the correct answer should appear. Also the score should appear out of how many card types have been guessed so far (e.g. if the player is on their 6th card type and haven't guessed it yet and they've got 3 correct so far, the score should read "Score: 3/5"). Can you give me the html, js and css for this app (include the css in the head of the html and the js methods as a separate file with calls to them via script tags in the html)

Copilot

A lightweight, GitHub Pages–friendly “Guess the FIFA Card Type” quiz

Below is a complete, vanilla HTML/CSS/JS app. It assumes your images live in a folder named cards at the repo root, and the correct answer is the image filename without its extension (normalized so dashes/underscores/punctuation don’t matter).

- Scoring shows “Score: correct/attempted” and only increments “attempted” after you finish a card (exactly like your 3/5 example while on the 6th card).
- Cards appear in random order and reshuffle when you’ve seen them all.
- Case-insensitive matching, ignores dashes/underscores/punctuation, and collapses extra spaces.
- After Submit Guess, it reveals the correct answer and enables Next.

Files

index.html (CSS included in head; JS loaded via script tag)

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <title>Guess the FIFA Card Type</title>

  <style>
    :root {
      --bg: #0f172a;
      --panel: #111827;
      --text: #e5e7eb;
      --muted: #9ca3af;
      --accent: #22c55e;
      --accent-2: #ef4444;
      --accent-3: #3b82f6;
      --input-bg: #0b1220;
      --border: #1f2937;
    }

    * { box-sizing: border-box; }
    body {
      margin: 0;
      font-family: Inter, system-ui, -apple-system, Segoe UI, Roboto, Arial, "Helvetica Neue", sans-serif;
      background: radial-gradient(1200px 600px at 20% -10%, #0a0f1f, transparent),
                  radial-gradient(1000px 500px at 110% 0%, #141b2e, transparent),
                  var(--bg);
      color: var(--text);
      min-height: 100vh;
      display: grid;
```

```
  place-items: center;
  padding: 24px;
}

.app {
  width: 100%;
  max-width: 720px;
  background: linear-gradient(180deg, rgba(255,255,255,0.04), rgba(255,255,255,0.02));
  border: 1px solid var(--border);
  border-radius: 16px;
  padding: 20px 20px 16px;
  box-shadow: 0 10px 40px rgba(0,0,0,0.35), inset 0 1px 0 rgba(255,255,255,0.04);
  backdrop-filter: blur(6px);
}

header {
  display: flex;
  align-items: baseline;
  justify-content: space-between;
  gap: 12px;
  padding: 6px 4px 14px;
  border-bottom: 1px solid var(--border);
  margin-bottom: 16px;
}

h1 {
  font-size: 20px;
  font-weight: 700;
  letter-spacing: 0.2px;
  margin: 0;
}

.score {
  font-weight: 600;
  color: var(--muted);
}

.figure {
  display: grid;
  place-items: center;
  background: linear-gradient(180deg, rgba(255,255,255,0.06), rgba(255,255,255,0.03));
  border: 1px solid var(--border);
  border-radius: 14px;
  padding: 18px;
  min-height: 320px;
  margin-bottom: 16px;
}

.figure img {
  max-width: 100%;
  max-height: 420px;
  object-fit: contain;
  border-radius: 12px;
  border: 1px solid var(--border);
  background: #0a0f1a;
  box-shadow: 0 8px 24px rgba(0,0,0,0.3);
}

.controls {
  display: grid;
  grid-template-columns: 1fr auto;
  gap: 10px;
  align-items: center;
  margin-top: 8px;
```

```
}

.field {
  display: grid;
  gap: 8px;
}

label {
  font-size: 14px;
  color: var(--muted);
}

input[type="text"] {
  width: 100%;
  padding: 12px 14px;
  font-size: 16px;
  color: var(--text);
  background: var(--input-bg);
  border: 1px solid var(--border);
  border-radius: 10px;
  outline: none;
  transition: border-color 120ms ease;
}

input[type="text"]:focus {
  border-color: var(--accent-3);
  box-shadow: 0 0 0 3px rgba(59, 130, 246, 0.2);
}

.btn {
  appearance: none;
  border: 1px solid var(--border);
  border-radius: 10px;
  padding: 12px 14px;
  font-size: 15px;
  font-weight: 700;
  letter-spacing: 0.2px;
  cursor: pointer;
  transition: transform 80ms ease, box-shadow 150ms ease, background 150ms ease, color 150ms ease;
  color: white;
  background: linear-gradient(180deg, #2563eb, #1d4ed8);
  box-shadow: 0 6px 18px rgba(37, 99, 235, 0.32);
}

.btn:hover { transform: translateY(-1px); }
.btn:active { transform: translateY(0); }

.btn.secondary {
  background: linear-gradient(180deg, #334155, #1f2a37);
  color: #e5e7eb;
  box-shadow: none;
}

.row {
  display: flex;
  gap: 10px;
  margin-top: 10px;
}

.feedback {
  margin-top: 10px;
  padding: 12px 14px;
  border-radius: 10px;
  border: 1px dashed var(--border);
```

```
    background: rgba(255,255,255,0.03);
    font-weight: 600;
    display: none;
  }

  .feedback.show { display: block; }
  .feedback.ok {
    border-color: rgba(34, 197, 94, 0.35);
    background: rgba(34, 197, 94, 0.08);
    color: #a7f3d0;
  }
  .feedback.nope {
    border-color: rgba(239, 68, 68, 0.35);
    background: rgba(239, 68, 68, 0.08);
    color: #fecaca;
  }

  footer {
    margin-top: 14px;
    font-size: 13px;
    color: var(--muted);
    display: flex;
    justify-content: space-between;
    align-items: center;
  }

  .hint {
    opacity: 0.8;
  }

  @media (max-width: 560px) {
    .controls { grid-template-columns: 1fr; }
    .row { flex-direction: column; }
  }
 </style>
</head>
<body>
 <main class="app" role="main">
  <header>
   <h1>Guess the FIFA Card Type</h1>
   <div id="score" class="score" aria-live="polite">Score: 0/0</div>
  </header>

  <section class="figure" aria-label="Card to guess">
   <img id="cardImage" src="" alt="FIFA card type to guess" />
  </section>

  <form id="guessForm" class="controls" autocomplete="off">
   <div class="field">
    <label for="guessInput">Your guess (card type)</label>
    <input id="guessInput" name="guess" type="text" placeholder="e.g., Team of the Season" required />
   </div>
   <button id="submitBtn" class="btn" type="submit">Submit Guess</button>
  </form>

  <div id="feedback" class="feedback" aria-live="polite"></div>

  <div class="row">
   <button id="nextBtn" class="btn secondary" type="button" disabled>Next</button>
   <button id="resetBtn" class="btn secondary" type="button">Reset</button>
  </div>

  <footer>
   <div class="hint">Tip: Press Enter to submit. After checking, use Next to continue.</div>
```

```
    </footer>
  </main>

  <script src="app.js" defer></script>
</body>
</html>
```

---

app.js (vanilla JS; update the CARDS array with your filenames)

```
// Configuration
const IMG_BASE = "cards/"; // folder containing your images at repo root

// 1) Put your image filenames here (relative to IMG_BASE).
// 2) The correct answer is derived from the filename (without extension).
// 3) Dashes/underscores/punctuation are ignored, and comparison is case-insensitive.
const CARDS = [
  // Examples — replace with your actual files:
  // "gold.png",
  // "silver.jpg",
  // "team-of-the-week.png",
  // "team_of_the_season.jpeg",
  // "ones-to-watch.webp",
];

// Utility: Fisher–Yates shuffle
function shuffle(arr) {
  const a = arr.slice();
  for (let i = a.length - 1; i > 0; i--) {
    const j = Math.floor(Math.random() * (i + 1));
    [a[i], a[j]] = [a[j], a[i]];
  }
  return a;
}

// Utility: get filename without extension
function baseName(path) {
  const f = path.split("/").pop();
  return f.replace(/\.[a-z0-9]+$/i, "");
}

// Normalize strings for comparison
function normalize(s) {
  return String(s)
    .toLowerCase()
    .replace(/[_-]+/g, " ")
    .replace(/[^a-z0-9 ]+/g, " ")
    .replace(/\s+/g, " ")
    .trim();
}

// Make a nice title for display
function titleCase(s) {
  return s
    .split(" ")
    .filter(Boolean)
    .map(w => w[0] ? w[0].toUpperCase() + w.slice(1) : w)
    .join(" ");
}

// DOM elements
const imgEl = document.getElementById("cardImage");
const scoreEl = document.getElementById("score");
```

```
const formEl = document.getElementById("guessForm");
const inputEl = document.getElementById("guessInput");
const submitBtn = document.getElementById("submitBtn");
const feedbackEl = document.getElementById("feedback");
const nextBtn = document.getElementById("nextBtn");
const resetBtn = document.getElementById("resetBtn");

// State
let order = [];
let idx = 0;
let attempted = 0; // completed cards
let correct = 0;
let answered = false;
let lastWasCorrect = false;

// Initialize
function init() {
  if (!CARDS.length) {
    feedbackEl.className = "feedback show";
    feedbackEl.textContent = "Add your card image filenames to CARDS[] in app.js to begin.";
    submitBtn.disabled = true;
    nextBtn.disabled = true;
    return;
  }
  order = shuffle(CARDS);
  idx = 0;
  attempted = 0;
  correct = 0;
  answered = false;
  lastWasCorrect = false;
  updateScore();
  loadCurrentCard();
  clearFeedback();
  inputEl.value = "";
  inputEl.focus();
}

function updateScore() {
  scoreEl.textContent = `Score: ${correct}/${attempted}`;
}

function loadCurrentCard() {
  const filename = order[idx];
  imgEl.src = IMG_BASE + filename;
  imgEl.alt = "FIFA card type to guess";
}

function clearFeedback() {
  feedbackEl.className = "feedback";
  feedbackEl.textContent = "";
}

function showFeedback(isCorrect, expected) {
  feedbackEl.className = `feedback show ${isCorrect ? "ok" : "nope"}`;
  if (isCorrect) {
    feedbackEl.textContent = `Correct! It's ${titleCase(expected)}.`;
  } else {
    feedbackEl.textContent = `Not quite. Correct answer: ${titleCase(expected)}.`;
  }
}

// After checking a guess, move to next card
function goNext() {
  // Count the previous card into the score
```

```
  attempted += 1;
  if (lastWasCorrect) correct += 1;
  updateScore();

  // Advance
  idx += 1;
  if (idx >= order.length) {
    order = shuffle(CARDS);
    idx = 0;
  }
  loadCurrentCard();

  // Reset UI
  answered = false;
  lastWasCorrect = false;
  inputEl.value = "";
  inputEl.disabled = false;
  submitBtn.disabled = false;
  nextBtn.disabled = true;
  clearFeedback();
  inputEl.focus();
}

// Events
formEl.addEventListener("submit", (e) => {
  e.preventDefault();
  if (answered) return;

  const filename = order[idx];
  const expected = normalize(baseName(filename));
  const guess = normalize(inputEl.value);

  lastWasCorrect = guess.length > 0 && guess === expected;
  answered = true;

  showFeedback(lastWasCorrect, expected);
  inputEl.disabled = true;
  submitBtn.disabled = true;
  nextBtn.disabled = false;
  nextBtn.focus();
});

nextBtn.addEventListener("click", () => {
  if (!answered) return; // only proceed after a submission
  goNext();
});

resetBtn.addEventListener("click", () => {
  init();
});

// Keyboard hint: Enter submits (handled by form); after answer, Enter triggers Next
document.addEventListener("keydown", (e) => {
  if (e.key === "Enter" && answered && !nextBtn.disabled) {
    e.preventDefault();
    nextBtn.click();
  }
});

// Boot
init();
```

How to set it up on GitHub Pages

1. Put your images in a folder named cards at the repo root.
    - Example: cards/team-of-the-season.png, cards/ones-to-watch.jpg
2. Edit app.js and list those filenames in the CARDS array.
    - Example: const CARDS = ["team-of-the-season.png","ones-to-watch.jpg"];
3. Keep index.html and app.js at the repo root.
4. Enable GitHub Pages: Settings → Pages → Deploy from branch (e.g., main), Folder: /root. Save.
5. Wait for the Pages link to appear; open it to play.

If you want to avoid maintaining the list manually, you can add a small build step to generate the array (or a *cards.json* manifest) from the folder contents, but for GitHub Pages-only hosting, a simple hardcoded list is the most reliable. github.com github.com github.com github.com

References

(1) GitHub - Fifteen Game
(2) GitHub - Webix/sources/models/progressch.js
(3) GitHub - OneKey Desktop
(4) GitHub - leChaffouin/taquin.js

You

No so in app.js, the cards array should automatically consist of the filenames of the cards in that folder (the js should be able to read the folder and store the names of every file in the cards folder automatically)

Copilot

Auto-populating cards from your GitHub folder (no manual list)

We'll fetch the file names in your cards folder directly from the GitHub REST API at runtime (client-side), then use those names to run the quiz. This works great on GitHub Pages for public repos.

- Randomized order, scoring as "correct/attempted," and exact behavior you described.
- Case-insensitive matching; ignores dashes/underscores/punctuation.
- Caches the folder listing for a few hours to avoid API rate limits.
- Auto-detects owner/repo from your Pages URL when possible, but you can hardcode it.

Files

index.html (CSS in head; JS via script tag)

```
<!doctype html>
<html lang="en">
<head>
 <meta charset="utf-8" />
 <meta name="viewport" content="width=device-width, initial-scale=1" />
 <title>Guess the FIFA Card Type</title>

 <style>
  :root {
   --bg: #0f172a;
   --panel: #111827;
   --text: #e5e7eb;
   --muted: #9ca3af;
   --accent: #22c55e;
   --accent-2: #ef4444;
   --accent-3: #3b82f6;
   --input-bg: #0b1220;
   --border: #1f2937;
  }
  * { box-sizing: border-box; }
  body {
```

```
  margin: 0;
  font-family: Inter, system-ui, -apple-system, Segoe UI, Roboto, Arial, "Helvetica Neue", sans-serif;
  background: radial-gradient(1200px 600px at 20% -10%, #0a0f1f, transparent),
              radial-gradient(1000px 500px at 110% 0%, #141b2e, transparent),
              var(--bg);
  color: var(--text);
  min-height: 100vh;
  display: grid;
  place-items: center;
  padding: 24px;
}
.app {
  width: 100%;
  max-width: 720px;
  background: linear-gradient(180deg, rgba(255,255,255,0.04), rgba(255,255,255,0.02));
  border: 1px solid var(--border);
  border-radius: 16px;
  padding: 20px 20px 16px;
  box-shadow: 0 10px 40px rgba(0,0,0,0.35), inset 0 1px 0 rgba(255,255,255,0.04);
  backdrop-filter: blur(6px);
}
header {
  display: flex;
  align-items: baseline;
  justify-content: space-between;
  gap: 12px;
  padding: 6px 4px 14px;
  border-bottom: 1px solid var(--border);
  margin-bottom: 16px;
}
h1 {
  font-size: 20px;
  font-weight: 700;
  letter-spacing: 0.2px;
  margin: 0;
}
.score { font-weight: 600; color: var(--muted); }
.figure {
  display: grid;
  place-items: center;
  background: linear-gradient(180deg, rgba(255,255,255,0.06), rgba(255,255,255,0.03));
  border: 1px solid var(--border);
  border-radius: 14px;
  padding: 18px;
  min-height: 320px;
  margin-bottom: 16px;
  position: relative;
}
.figure img {
  max-width: 100%;
  max-height: 420px;
  object-fit: contain;
  border-radius: 12px;
  border: 1px solid var(--border);
  background: #0a0f1a;
  box-shadow: 0 8px 24px rgba(0,0,0,0.3);
}
.loading {
  position: absolute;
  inset: 0;
  display: grid;
  place-items: center;
  color: var(--muted);
  font-weight: 600;
```

```
      background: rgba(0,0,0,0.08);
      border-radius: 12px;
      backdrop-filter: blur(1px);
    }
    .controls {
      display: grid;
      grid-template-columns: 1fr auto;
      gap: 10px;
      align-items: center;
      margin-top: 8px;
    }
    .field { display: grid; gap: 8px; }
    label { font-size: 14px; color: var(--muted); }
    input[type="text"] {
      width: 100%; padding: 12px 14px; font-size: 16px; color: var(--text);
      background: var(--input-bg); border: 1px solid var(--border);
      border-radius: 10px; outline: none; transition: border-color 120ms ease;
    }
    input[type="text"]:focus {
      border-color: var(--accent-3);
      box-shadow: 0 0 0 3px rgba(59,130,246,0.2);
    }
    .btn {
      appearance: none; border: 1px solid var(--border); border-radius: 10px;
      padding: 12px 14px; font-size: 15px; font-weight: 700; letter-spacing: 0.2px;
      cursor: pointer; transition: transform 80ms ease, box-shadow 150ms ease, background 150ms ease, color 150ms
ease;
      color: white; background: linear-gradient(180deg, #2563eb, #1d4ed8);
      box-shadow: 0 6px 18px rgba(37,99,235,0.32);
    }
    .btn:hover { transform: translateY(-1px); }
    .btn:active { transform: translateY(0); }
    .btn.secondary {
      background: linear-gradient(180deg, #334155, #1f2a37);
      color: #e5e7eb; box-shadow: none;
    }
    .row { display: flex; gap: 10px; margin-top: 10px; }
    .feedback {
      margin-top: 10px; padding: 12px 14px; border-radius: 10px;
      border: 1px dashed var(--border); background: rgba(255,255,255,0.03);
      font-weight: 600; display: none;
    }
    .feedback.show { display: block; }
    .feedback.ok {
      border-color: rgba(34,197,94,0.35); background: rgba(34,197,94,0.08); color: #a7f3d0;
    }
    .feedback.nope {
      border-color: rgba(239,68,68,0.35); background: rgba(239,68,68,0.08); color: #fecaca;
    }
    footer {
      margin-top: 14px; font-size: 13px; color: var(--muted);
      display: flex; justify-content: space-between; align-items: center;
    }
    .hint { opacity: 0.8; }
    @media (max-width: 560px) {
      .controls { grid-template-columns: 1fr; }
      .row { flex-direction: column; }
    }
  </style>
</head>
<body>
  <main class="app" role="main">
    <header>
      <h1>Guess the FIFA Card Type</h1>
```

```
    <div id="score" class="score" aria-live="polite">Score: 0/0</div>
  </header>

  <section class="figure" aria-label="Card to guess">
    <img id="cardImage" src="" alt="FIFA card type to guess" />
    <div id="loading" class="loading">Loading cards…</div>
  </section>

  <form id="guessForm" class="controls" autocomplete="off">
    <div class="field">
      <label for="guessInput">Your guess (card type)</label>
      <input id="guessInput" name="guess" type="text" placeholder="e.g., Team of the Season" required />
    </div>
    <button id="submitBtn" class="btn" type="submit">Submit Guess</button>
  </form>

  <div id="feedback" class="feedback" aria-live="polite"></div>

  <div class="row">
    <button id="nextBtn" class="btn secondary" type="button" disabled>Next</button>
    <button id="resetBtn" class="btn secondary" type="button">Reset</button>
  </div>

  <footer>
    <div class="hint">Tip: Press Enter to submit. After checking, use Next to continue.</div>
    <div id="sourceInfo" class="hint"></div>
  </footer>
 </main>

 <script src="app.js" defer></script>
</body>
</html>
```

---

app.js (auto-fetch filenames from your cards folder via GitHub API)

```
// --- Configuration ------------------------------------------------------
// If your site is at https://<owner>.github.io/<repo>/, detection fills these in.
// If using a custom domain or it fails, set OWNER, REPO, and BRANCH manually.
const CONFIG = {
  OWNER: null,      // e.g., "hasan-dev"
  REPO: null,       // e.g., "fifa-card-quiz"
  BRANCH: "main",   // or "gh-pages" or whatever branch your Pages serves from
  CARDS_PATH: "cards", // folder containing your images relative to repo root
  CACHE_TTL_MS: 6 * 60 * 60 * 1000 // 6 hours
};

// Optional: force using GitHub API download_url for images (usually not needed)
// If false, images are loaded from your Pages site via relative paths (faster).
const USE_DOWNLOAD_URLS = false;

// --- Helpers: repo detection and API URL -------------------------------
function detectRepoContext() {
  const host = window.location.hostname;
  const pathParts = window.location.pathname.split("/").filter(Boolean);
  const m = host.match(/^([^.]+)\.github\.io$/);
  const owner = CONFIG.OWNER || (m ? m[1] : null);
  // On project pages, first path segment is repo; on user/org pages it's absent.
  const repo = CONFIG.REPO || (m && pathParts.length ? pathParts[0] : null);
  const branch = CONFIG.BRANCH || "main";
  return { owner, repo, branch };
}
```

```
function ghContentsUrl(owner, repo, cardsPath, branch) {
  const encPath = encodeURIComponent(cardsPath).replace(/%2F/g, "/");
  const encRef = encodeURIComponent(branch);
  return `https://api.github.com/repos/${owner}/${repo}/contents/${encPath}?ref=${encRef}`;
}

// --- Storage cache ------------------------------------------------------
function cacheKey({ owner, repo, branch }, path) {
  return `cards_manifest::${owner || "?"}::${repo || "?"}::${branch}::${path}`;
}

function readCache(key, ttlMs) {
  try {
    const raw = localStorage.getItem(key);
    if (!raw) return null;
    const { ts, items } = JSON.parse(raw);
    if (!ts || !Array.isArray(items)) return null;
    if (Date.now() - ts > ttlMs) return null;
    return items;
  } catch {
    return null;
  }
}

function writeCache(key, items) {
  try {
    localStorage.setItem(key, JSON.stringify({ ts: Date.now(), items }));
  } catch {
    // ignore quota errors
  }
}

function clearCache(key) {
  try { localStorage.removeItem(key); } catch {}
}

// --- Filename utilities -------------------------------------------------
function baseName(path) {
  const f = path.split("/").pop();
  return f.replace(/\.[a-z0-9]+$/i, "");
}

function normalize(s) {
  return String(s)
    .toLowerCase()
    .replace(/[_-]+/g, " ")
    .replace(/[^a-z0-9 ]+/g, " ")
    .replace(/\s+/g, " ")
    .trim();
}

function titleCase(s) {
  return s
    .split(" ")
    .filter(Boolean)
    .map(w => w[0] ? w[0].toUpperCase() + w.slice(1) : w)
    .join(" ");
}

function isImageName(name) {
  return /\.(png|jpe?g|webp|gif|svg)$/i.test(name);
}

// --- Fetch folder listing from GitHub -----------------------------------
```

```
async function fetchCardFilenames() {
  const ctx = detectRepoContext();
  const { owner, repo, branch } = ctx;

  const infoEl = document.getElementById("sourceInfo");
  if (!owner || !repo) {
    infoEl.textContent = "Set OWNER/REPO in app.js (auto-detect failed).";
  } else {
    infoEl.textContent = `${owner}/${repo}@${branch} • ${CONFIG.CARDS_PATH}`;
  }

  const cKey = cacheKey(ctx, CONFIG.CARDS_PATH);
  const cached = readCache(cKey, CONFIG.CACHE_TTL_MS);
  if (cached && cached.length) return cached;

  if (!owner || !repo) throw new Error("Missing owner or repo to query GitHub API.");

  const url = ghContentsUrl(owner, repo, CONFIG.CARDS_PATH, branch);
  const resp = await fetch(url, {
    headers: { "Accept": "application/vnd.github.v3+json" }
  });
  if (!resp.ok) {
    throw new Error(`GitHub API error ${resp.status}`);
  }
  const json = await resp.json();
  if (!Array.isArray(json)) throw new Error("Unexpected API response.");
  const files = json
    .filter(item => item && item.type === "file" && isImageName(item.name))
    .map(item => ({
      name: item.name,
      download_url: item.download_url
    }));

  const names = files.map(f => f.name);
  writeCache(cKey, USE_DOWNLOAD_URLS ? files.map(f => f.download_url) : names);
  return USE_DOWNLOAD_URLS ? files.map(f => f.download_url) : names;
}

// --- UI/Quiz logic -------------------------------------------------------
const IMG_BASE = "cards/"; // used when not using download_url
const imgEl = document.getElementById("cardImage");
const loadingEl = document.getElementById("loading");
const scoreEl = document.getElementById("score");
const formEl = document.getElementById("guessForm");
const inputEl = document.getElementById("guessInput");
const submitBtn = document.getElementById("submitBtn");
const feedbackEl = document.getElementById("feedback");
const nextBtn = document.getElementById("nextBtn");
const resetBtn = document.getElementById("resetBtn");

let ORDER = [];     // array of filenames or full URLs
let IDX = 0;
let ATTEMPTED = 0;  // completed cards
let CORRECT = 0;
let ANSWERED = false;
let LAST_OK = false;

function shuffle(arr) {
  const a = arr.slice();
  for (let i = a.length - 1; i > 0; i--) {
    const j = Math.floor(Math.random() * (i + 1));
    [a[i], a[j]] = [a[j], a[i]];
  }
  return a;
```

```
}

function updateScore() {
  scoreEl.textContent = `Score: ${CORRECT}/${ATTEMPTED}`;
}

function clearFeedback() {
  feedbackEl.className = "feedback";
  feedbackEl.textContent = "";
}

function showFeedback(isCorrect, expected) {
  feedbackEl.className = `feedback show ${isCorrect ? "ok" : "nope"}`;
  feedbackEl.textContent = isCorrect
    ? `Correct! It's ${titleCase(expected)}.`
    : `Not quite. Correct answer: ${titleCase(expected)}.`;
}

function currentDisplayName(entry) {
  // entry is either "filename.ext" or a full download_url
  const name = entry.includes("://")
    ? entry.split("/").pop().split("?")[0]
    : entry;
  return baseName(name);
}

function loadCurrentCard() {
  const entry = ORDER[IDX];
  if (!entry) return;
  imgEl.src = entry.includes("://") ? entry : (IMG_BASE + entry);
  imgEl.alt = "FIFA card type to guess";
}

async function startGame() {
  try {
    loadingEl.style.display = "grid";
    submitBtn.disabled = true;
    nextBtn.disabled = true;
    inputEl.disabled = true;

    const items = await fetchCardFilenames();
    if (!items.length) {
      throw new Error("No image files found in cards folder.");
    }

    ORDER = shuffle(items);
    IDX = 0; ATTEMPTED = 0; CORRECT = 0; ANSWERED = false; LAST_OK = false;
    updateScore();
    loadCurrentCard();
    clearFeedback();

    inputEl.value = "";
    inputEl.disabled = false;
    submitBtn.disabled = false;
    loadingEl.style.display = "none";
    inputEl.focus();
  } catch (err) {
    loadingEl.style.display = "none";
    clearFeedback();
    feedbackEl.className = "feedback show nope";
    feedbackEl.textContent = `Couldn't load cards automatically: ${err.message}.
You can set OWNER/REPO/BRANCH in app.js or try again.`;
    console.error(err);
  }
```

```
}

function goNext() {
  ATTEMPTED += 1;
  if (LAST_OK) CORRECT += 1;
  updateScore();

  IDX += 1;
  if (IDX >= ORDER.length) {
    ORDER = shuffle(ORDER);
    IDX = 0;
  }
  loadCurrentCard();

  ANSWERED = false;
  LAST_OK = false;
  inputEl.value = "";
  inputEl.disabled = false;
  submitBtn.disabled = false;
  nextBtn.disabled = true;
  clearFeedback();
  inputEl.focus();
}

// Events
formEl.addEventListener("submit", (e) => {
  e.preventDefault();
  if (ANSWERED) return;

  const entry = ORDER[IDX];
  const expected = normalize(currentDisplayName(entry));
  const guess = normalize(inputEl.value);

  LAST_OK = guess.length > 0 && guess === expected;
  ANSWERED = true;

  showFeedback(LAST_OK, expected);
  inputEl.disabled = true;
  submitBtn.disabled = true;
  nextBtn.disabled = false;
  nextBtn.focus();
});

nextBtn.addEventListener("click", () => {
  if (!ANSWERED) return;
  goNext();
});

resetBtn.addEventListener("click", () => {
  const ctx = detectRepoContext();
  clearCache(cacheKey(ctx, CONFIG.CARDS_PATH));
  startGame();
});

// Enter key goes Next after answering
document.addEventListener("keydown", (e) => {
  if (e.key === "Enter" && ANSWERED && !nextBtn.disabled) {
    e.preventDefault();
    nextBtn.click();
  }
});

// Boot
startGame();
```

Configure and deploy on GitHub Pages

1. Create your cards folder at the repo root and put your images inside it.
    - Example: cards/team-of-the-season.png, cards/ones-to-watch.jpg
2. Ensure your repo is public (so the GitHub API can list the folder without a token).
3. If your site is at https://yourname.github.io/yourrepo/, it will auto-detect owner/repo.
    - If you use a custom domain or detection fails, open app.js and set:
        - CONFIG.OWNER = "yourname"
        - CONFIG.REPO = "yourrepo"
        - CONFIG.BRANCH = "main" (or gh-pages, etc., matching your Pages branch)
4. Commit index.html, app.js, and the cards/ folder to the same branch you publish.
5. Enable GitHub Pages: Settings → Pages → Deploy from branch → select the branch and /root.
6. Visit your Pages URL. The app fetches the folder listing, shuffles, and starts the quiz.

Notes and options

- If you prefer not to use the GitHub API, add a cards.json manifest at the repo root and fetch that instead. But the API approach keeps everything automatic.
- To tweak strictness, adjust the normalize function (e.g., allow partial matches or synonyms).
- If you hit rate limits while testing, the cache reduces calls; you can also increase CACHE_TTL_MS. github.com github.com github.com

References

(1) GitHub - Fifteen Game
(2) GitHub - Webix/sources/models/progressch.js
(3) GitHub - OneKey Desktop

You

Ok so my public GitHub repo is "https://github.com/HasanBradfordUni/myPublicFiles/tree/main" but my GitHub pages is configured to use the custom domain "http://akhtarhasansoftwaresolutions.tech/", can you give me just the updated app.js considering this?

Copilot

Absolutely, Hasan! Since your GitHub Pages uses a custom domain (http://akhtarhasansoftwaresolutions.tech/), auto-detection won't work, so we'll hardcode the necessary repo details in your app.js. Here's the **updated configuration section only** — just replace the existing CONFIG block near the top of your app.js file with this:

```
// --- Configuration ------------------------------------------------------
const CONFIG = {
  OWNER: "HasanBradfordUni",          // GitHub username
  REPO: "myPublicFiles",              // Repository name
  BRANCH: "main",                     // Branch GitHub Pages is publishing from
  CARDS_PATH: "cards",                // Path to your folder containing card images
  CACHE_TTL_MS: 6 * 60 * 60 * 1000    // 6-hour cache timeout for API results
};

// Optional: force using GitHub API image URLs (vs. local relative paths)
// If your custom domain maps everything correctly, you can leave this as false.
// Otherwise, you can set it to true if images fail to load.
const USE_DOWNLOAD_URLS = false;
```

Now the rest of the app will use the GitHub API to list image files from the cards folder inside your repo, randomize them, and display the quiz automatically — all under your custom domain.

Let me know if you'd like the app to support offline fallback or a manifest file for additional optimization!

You

(Now Wednesday, Aug 13)
I get this error in the javascript:
Uncaught (in promise) TypeError: can't access property "style", loadingEl is null
startGame https://akhtarhasansoftwaresolutions.tech/app.js:211
<anonymous> https://akhtarhasansoftwaresolutions.tech/app.js:269

Specifically in this method:

```
async function startGame() {
  try {
    loadingEl.style.display = "grid";
    submitBtn.disabled = true;
    nextBtn.disabled = true;
    inputEl.disabled = true;

    const items = await fetchCardFilenames();
    if (!items.length) {
      throw new Error("No image files found in cards folder.");
    }

    ORDER = shuffle(items);
    IDX = 0; ATTEMPTED = 0; CORRECT = 0; ANSWERED = false; LAST_OK = false;
    updateScore();
    loadCurrentCard();
    clearFeedback();

    inputEl.value = "";
    inputEl.disabled = false;
    submitBtn.disabled = false;
    loadingEl.style.display = "none";
    inputEl.focus();
  } catch (err) {
    loadingEl.style.display = "none";
    clearFeedback();
    feedbackEl.className = "feedback show nope";
    feedbackEl.textContent = `Couldn't load cards automatically: ${err.message}.
You can set OWNER/REPO/BRANCH in app.js or try again.`;
    console.error(err);
  }
}
```

Copilot

Thanks for the detailed error report, Hasan — this is a classic case of a missing DOM element. The error means that *loadingEl* is *null*, which happens when the element with *id="loading"* isn't present in your *index.html*.

☑ Fix: Add the missing loading overlay

Make sure your HTML includes this inside the .figure section:

```
<section class="figure" aria-label="Card to guess">
  <img id="cardImage" src="" alt="FIFA card type to guess" />
  <div id="loading" class="loading">Loading cards…</div>
</section>
```

That is what loadingEl refers to in your app.js. Without it, the script throws an error when trying to access .style.

🛠 Optional: Defensive fix in JS

If you want to make your JS more robust in case the element is missing, you can wrap the *loadingEl.style* calls like this:

```
if (loadingEl) loadingEl.style.display = "grid";
// ...
if (loadingEl) loadingEl.style.display = "none";
```

But ideally, the element should always be present — it's part of the visual feedback while the app loads your card list.

Let me know if you'd like a fallback loader or a smoother transition while fetching the card list. I can also help you add a spinner or animation if you want to polish the UX.

You

Now I get this message:
Couldn't load cards automatically: can't access property "textContent", infoEl is null. You can set OWNER/REPO/BRANCH in app.js or try again.

And this in the console:

```
TypeError: can't access property "textContent", infoEl is null
  fetchCardFilenames https://akhtarhasansoftwaresolutions.tech/app.js:98
  startGame https://akhtarhasansoftwaresolutions.tech/app.js:194
  <anonymous> https://akhtarhasansoftwaresolutions.tech/app.js:281
```

Copilot

You're on the right track, Hasan — this is the same kind of issue as before: infoEl is null because the corresponding element is missing from your HTML.

☑ Fix: Add the missing infoEl element

In your app.js, you're likely doing something like:

```
infoEl.textContent = `Found ${files.length} cards in folder.`;
```

So you need to make sure your HTML includes:

```
<p id="info" class="info"></p>
```

You can place it somewhere logical in your layout — for example, just below the score or above the card image:

```
<header>
  <h1>FIFA Card Quiz</h1>
  <p id="info" class="info"></p>
  <p id="score" class="score">Score: 0/0</p>
</header>
```

🛠 Optional: Defensive coding in JS

If you want to avoid future crashes from missing elements, you can always check before accessing properties:

```
if (infoEl) {
  infoEl.textContent = `Found ${files.length} cards in folder.`;
}
```

But again, since this is part of your core UI, it's better to ensure the element is always present.

Let me know if you want to auto-hide that info message after a few seconds or style it differently based on success/failure. I can help you make it more dynamic or visually engaging.

You

This is app.js currently, can you remove all logic that automatically detects the repo details and keep my config at the top, using only that:

```
// --- Configuration ------------------------------------------------------------
const CONFIG = {
  OWNER: "HasanBradfordUni",          // GitHub username
  REPO: "myPublicFiles",              // Repository name
  BRANCH: "main",                     // Branch GitHub Pages is publishing from
  CARDS_PATH: "cards",                // Path to your folder containing card images
  CACHE_TTL_MS: 6 * 60 * 60 * 1000    // 6-hour cache timeout for API results
};

// Optional: force using GitHub API download_url for images (usually not needed)
// If false, images are loaded from your Pages site via relative paths (faster).
const USE_DOWNLOAD_URLS = false;

// --- Helpers: repo detection and API URL --------------------------------------
function detectRepoContext() {
  const host = window.location.hostname;
  const pathParts = window.location.pathname.split("/").filter(Boolean);
  const m = host.match(/^([^.]+)\.github\.io$/);
  const owner = CONFIG.OWNER || (m ? m[1] : null);
  // On project pages, first path segment is repo; on user/org pages it's absent.
  const repo = CONFIG.REPO || (m && pathParts.length ? pathParts[0] : null);
  const branch = CONFIG.BRANCH || "main";
  return { owner, repo, branch };
}

function ghContentsUrl(owner, repo, cardsPath, branch) {
  const encPath = encodeURIComponent(cardsPath).replace(/%2F/g, "/");
  const encRef = encodeURIComponent(branch);
  return `https://api.github.com/repos/${owner}/${repo}/contents/${encPath}?ref=${encRef}`;
}

// --- Storage cache ------------------------------------------------------------
function cacheKey({ owner, repo, branch }, path) {
  return `cards_manifest::${owner || "?"}::${repo || "?"}::${branch}::${path}`;
}

function readCache(key, ttlMs) {
  try {
    const raw = localStorage.getItem(key);
    if (!raw) return null;
    const { ts, items } = JSON.parse(raw);
    if (!ts || !Array.isArray(items)) return null;
    if (Date.now() - ts > ttlMs) return null;
    return items;
  } catch {
    return null;
  }
}

function writeCache(key, items) {
  try {
    localStorage.setItem(key, JSON.stringify({ ts: Date.now(), items }));
  } catch {
    // ignore quota errors
  }
}

function clearCache(key) {
  try { localStorage.removeItem(key); } catch {}
}
```

```
// --- Filename utilities -----------------------------------------------------
function baseName(path) {
  const f = path.split("/").pop();
  return f.replace(/\.[a-z0-9]+$/i, "");
}

function normalize(s) {
  return String(s)
    .toLowerCase()
    .replace(/[_-]+/g, " ")
    .replace(/[^a-z0-9 ]+/g, " ")
    .replace(/\s+/g, " ")
    .trim();
}

function titleCase(s) {
  return s
    .split(" ")
    .filter(Boolean)
    .map(w => w[0] ? w[0].toUpperCase() + w.slice(1) : w)
    .join(" ");
}

function isImageName(name) {
  return /\.(png|jpe?g|webp|gif|svg)$/i.test(name);
}

// --- Fetch folder listing from GitHub ---------------------------------------
async function fetchCardFilenames() {
  const ctx = detectRepoContext();
  const { owner, repo, branch } = ctx;

  const infoEl = document.getElementById("sourceInfo");
  if (!owner || !repo) {
    infoEl.textContent = "Set OWNER/REPO in app.js (auto-detect failed).";
  } else {
    infoEl.textContent = `${owner}/${repo}@${branch} • ${CONFIG.CARDS_PATH}`;
  }

  const cKey = cacheKey(ctx, CONFIG.CARDS_PATH);
  const cached = readCache(cKey, CONFIG.CACHE_TTL_MS);
  if (cached && cached.length) return cached;

  if (!owner || !repo) throw new Error("Missing owner or repo to query GitHub API.");

  const url = ghContentsUrl(owner, repo, CONFIG.CARDS_PATH, branch);
  const resp = await fetch(url, {
    headers: { "Accept": "application/vnd.github.v3+json" }
  });
  if (!resp.ok) {
    throw new Error(`GitHub API error ${resp.status}`);
  }
  const json = await resp.json();
  if (!Array.isArray(json)) throw new Error("Unexpected API response.");
  const files = json
    .filter(item => item && item.type === "file" && isImageName(item.name))
    .map(item => ({
      name: item.name,
      download_url: item.download_url
    }));

  const names = files.map(f => f.name);
  writeCache(cKey, USE_DOWNLOAD_URLS ? files.map(f => f.download_url) : names);
```

```
  return USE_DOWNLOAD_URLS ? files.map(f => f.download_url) : names;
}

// --- UI/Quiz logic ------------------------------------------------------
const IMG_BASE = "cards/"; // used when not using download_url
const imgEl = document.getElementById("cardImage");
const loadingEl = document.getElementById("loading");
const scoreEl = document.getElementById("score");
const formEl = document.getElementById("guessForm");
const inputEl = document.getElementById("guessInput");
const submitBtn = document.getElementById("submitBtn");
const feedbackEl = document.getElementById("feedback");
const nextBtn = document.getElementById("nextBtn");
const resetBtn = document.getElementById("resetBtn");

let ORDER = [];     // array of filenames or full URLs
let IDX = 0;
let ATTEMPTED = 0;  // completed cards
let CORRECT = 0;
let ANSWERED = false;
let LAST_OK = false;

function shuffle(arr) {
  const a = arr.slice();
  for (let i = a.length - 1; i > 0; i--) {
    const j = Math.floor(Math.random() * (i + 1));
    [a[i], a[j]] = [a[j], a[i]];
  }
  return a;
}

function updateScore() {
  scoreEl.textContent = `Score: ${CORRECT}/${ATTEMPTED}`;
}

function clearFeedback() {
  feedbackEl.className = "feedback";
  feedbackEl.textContent = "";
}

function showFeedback(isCorrect, expected) {
  feedbackEl.className = `feedback show ${isCorrect ? "ok" : "nope"}`;
  feedbackEl.textContent = isCorrect
    ? `Correct! It's ${titleCase(expected)}.`
    : `Not quite. Correct answer: ${titleCase(expected)}.`;
}

function currentDisplayName(entry) {
  // entry is either "filename.ext" or a full download_url
  const name = entry.includes("://")
    ? entry.split("/").pop().split("?")[0]
    : entry;
  return baseName(name);
}

function loadCurrentCard() {
  const entry = ORDER[IDX];
  if (!entry) return;
  imgEl.src = entry.includes("://") ? entry : (IMG_BASE + entry);
  imgEl.alt = "FIFA card type to guess";
}

async function startGame() {
  try {
```

```
    loadingEl.style.display = "grid";
    submitBtn.disabled = true;
    nextBtn.disabled = true;
    inputEl.disabled = true;

    const items = await fetchCardFilenames();
    if (!items.length) {
      throw new Error("No image files found in cards folder.");
    }

    ORDER = shuffle(items);
    IDX = 0; ATTEMPTED = 0; CORRECT = 0; ANSWERED = false; LAST_OK = false;
    updateScore();
    loadCurrentCard();
    clearFeedback();

    inputEl.value = "";
    inputEl.disabled = false;
    submitBtn.disabled = false;
    loadingEl.style.display = "none";
    inputEl.focus();
  } catch (err) {
    loadingEl.style.display = "none";
    clearFeedback();
    feedbackEl.className = "feedback show nope";
    feedbackEl.textContent = `Couldn't load cards automatically: ${err.message}.
 You can set OWNER/REPO/BRANCH in app.js or try again.`;
    console.error(err);
  }
}

function goNext() {
  ATTEMPTED += 1;
  if (LAST_OK) CORRECT += 1;
  updateScore();

  IDX += 1;
  if (IDX >= ORDER.length) {
    ORDER = shuffle(ORDER);
    IDX = 0;
  }
  loadCurrentCard();

  ANSWERED = false;
  LAST_OK = false;
  inputEl.value = "";
  inputEl.disabled = false;
  submitBtn.disabled = false;
  nextBtn.disabled = true;
  clearFeedback();
  inputEl.focus();
}

// Events
formEl.addEventListener("submit", (e) => {
  e.preventDefault();
  if (ANSWERED) return;

  const entry = ORDER[IDX];
  const expected = normalize(currentDisplayName(entry));
  const guess = normalize(inputEl.value);

  LAST_OK = guess.length > 0 && guess === expected;
  ANSWERED = true;
```

```
  showFeedback(LAST_OK, expected);
  inputEl.disabled = true;
  submitBtn.disabled = true;
  nextBtn.disabled = false;
  nextBtn.focus();
});

nextBtn.addEventListener("click", () => {
  if (!ANSWERED) return;
  goNext();
});

resetBtn.addEventListener("click", () => {
  const ctx = detectRepoContext();
  clearCache(cacheKey(ctx, CONFIG.CARDS_PATH));
  startGame();
});

// Enter key goes Next after answering
document.addEventListener("keydown", (e) => {
  if (e.key === "Enter" && ANSWERED && !nextBtn.disabled) {
    e.preventDefault();
    nextBtn.click();
  }
});

// Boot
startGame();
```

Copilot

Here's your **cleaned-up app.js** with all auto-detection logic removed. It now uses only the hardcoded CONFIG values you provided, and all references to *detectRepoContext*() have been eliminated.

☑ Updated app.js (fully manual config, no auto-detection)

```
// --- Configuration ------------------------------------------------------
const CONFIG = {
  OWNER: "HasanBradfordUni",
  REPO: "myPublicFiles",
  BRANCH: "main",
  CARDS_PATH: "cards",
  CACHE_TTL_MS: 6 * 60 * 60 * 1000
};

const USE_DOWNLOAD_URLS = false;

// --- Helpers ------------------------------------------------------------
function ghContentsUrl(owner, repo, cardsPath, branch) {
  const encPath = encodeURIComponent(cardsPath).replace(/%2F/g, "/");
  const encRef = encodeURIComponent(branch);
  return `https://api.github.com/repos/${owner}/${repo}/contents/${encPath}?ref=${encRef}`;
}

function cacheKey(path) {
  return `cards_manifest::${CONFIG.OWNER}::${CONFIG.REPO}::${CONFIG.BRANCH}::${path}`;
}

function readCache(key, ttlMs) {
  try {
    const raw = localStorage.getItem(key);
    if (!raw) return null;
```

```
    const { ts, items } = JSON.parse(raw);
    if (!ts || !Array.isArray(items)) return null;
    if (Date.now() - ts > ttlMs) return null;
    return items;
  } catch {
    return null;
  }
}

function writeCache(key, items) {
  try {
    localStorage.setItem(key, JSON.stringify({ ts: Date.now(), items }));
  } catch {}
}

function clearCache(key) {
  try { localStorage.removeItem(key); } catch {}
}

function baseName(path) {
  const f = path.split("/").pop();
  return f.replace(/\.[a-z0-9]+$/i, "");
}

function normalize(s) {
  return String(s).toLowerCase().replace(/[_-]+/g, " ").replace(/[^a-z0-9 ]+/g, " ").replace(/\s+/g, " ").trim();
}

function titleCase(s) {
  return s.split(" ").filter(Boolean).map(w => w[0] ? w[0].toUpperCase() + w.slice(1) : w).join(" ");
}

function isImageName(name) {
  return /\.(png|jpe?g|webp|gif|svg)$/i.test(name);
}

// --- Fetch folder listing from GitHub ---------------------------------------
async function fetchCardFilenames() {
  const infoEl = document.getElementById("sourceInfo");
  if (infoEl) {
    infoEl.textContent = `${CONFIG.OWNER}/${CONFIG.REPO}@${CONFIG.BRANCH} •
${CONFIG.CARDS_PATH}`;
  }

  const cKey = cacheKey(CONFIG.CARDS_PATH);
  const cached = readCache(cKey, CONFIG.CACHE_TTL_MS);
  if (cached && cached.length) return cached;

  const url = ghContentsUrl(CONFIG.OWNER, CONFIG.REPO, CONFIG.CARDS_PATH, CONFIG.BRANCH);
  const resp = await fetch(url, {
    headers: { "Accept": "application/vnd.github.v3+json" }
  });
  if (!resp.ok) throw new Error(`GitHub API error ${resp.status}`);
  const json = await resp.json();
  if (!Array.isArray(json)) throw new Error("Unexpected API response.");

  const files = json
    .filter(item => item && item.type === "file" && isImageName(item.name))
    .map(item => ({
      name: item.name,
      download_url: item.download_url
    }));

  const names = files.map(f => f.name);
```

```
  writeCache(cKey, USE_DOWNLOAD_URLS ? files.map(f => f.download_url) : names);
  return USE_DOWNLOAD_URLS ? files.map(f => f.download_url) : names;
}

// --- UI/Quiz logic ------------------------------------------------------------
const IMG_BASE = "cards/";
const imgEl = document.getElementById("cardImage");
const loadingEl = document.getElementById("loading");
const scoreEl = document.getElementById("score");
const formEl = document.getElementById("guessForm");
const inputEl = document.getElementById("guessInput");
const submitBtn = document.getElementById("submitBtn");
const feedbackEl = document.getElementById("feedback");
const nextBtn = document.getElementById("nextBtn");
const resetBtn = document.getElementById("resetBtn");

let ORDER = [];
let IDX = 0;
let ATTEMPTED = 0;
let CORRECT = 0;
let ANSWERED = false;
let LAST_OK = false;

function shuffle(arr) {
  const a = arr.slice();
  for (let i = a.length - 1; i > 0; i--) {
    const j = Math.floor(Math.random() * (i + 1));
    [a[i], a[j]] = [a[j], a[i]];
  }
  return a;
}

function updateScore() {
  scoreEl.textContent = `Score: ${CORRECT}/${ATTEMPTED}`;
}

function clearFeedback() {
  feedbackEl.className = "feedback";
  feedbackEl.textContent = "";
}

function showFeedback(isCorrect, expected) {
  feedbackEl.className = `feedback show ${isCorrect ? "ok" : "nope"}`;
  feedbackEl.textContent = isCorrect
    ? `Correct! It's ${titleCase(expected)}.`
    : `Not quite. Correct answer: ${titleCase(expected)}.`;
}

function currentDisplayName(entry) {
  const name = entry.includes("://")
    ? entry.split("/").pop().split("?")[0]
    : entry;
  return baseName(name);
}

function loadCurrentCard() {
  const entry = ORDER[IDX];
  if (!entry) return;
  imgEl.src = entry.includes("://") ? entry : (IMG_BASE + entry);
  imgEl.alt = "FIFA card type to guess";
}

async function startGame() {
  try {
```

```
    if (loadingEl) loadingEl.style.display = "grid";
    submitBtn.disabled = true;
    nextBtn.disabled = true;
    inputEl.disabled = true;

    const items = await fetchCardFilenames();
    if (!items.length) throw new Error("No image files found in cards folder.");

    ORDER = shuffle(items);
    IDX = 0; ATTEMPTED = 0; CORRECT = 0; ANSWERED = false; LAST_OK = false;
    updateScore();
    loadCurrentCard();
    clearFeedback();

    inputEl.value = "";
    inputEl.disabled = false;
    submitBtn.disabled = false;
    if (loadingEl) loadingEl.style.display = "none";
    inputEl.focus();
  } catch (err) {
    if (loadingEl) loadingEl.style.display = "none";
    clearFeedback();
    feedbackEl.className = "feedback show nope";
    feedbackEl.textContent = `Couldn't load cards automatically: ${err.message}.`;
    console.error(err);
  }
}

function goNext() {
  ATTEMPTED += 1;
  if (LAST_OK) CORRECT += 1;
  updateScore();

  IDX += 1;
  if (IDX >= ORDER.length) {
    ORDER = shuffle(ORDER);
    IDX = 0;
  }
  loadCurrentCard();

  ANSWERED = false;
  LAST_OK = false;
  inputEl.value = "";
  inputEl.disabled = false;
  submitBtn.disabled = false;
  nextBtn.disabled = true;
  clearFeedback();
  inputEl.focus();
}

// Events
formEl.addEventListener("submit", (e) => {
  e.preventDefault();
  if (ANSWERED) return;

  const entry = ORDER[IDX];
  const expected = normalize(currentDisplayName(entry));
  const guess = normalize(inputEl.value);

  LAST_OK = guess.length > 0 && guess === expected;
  ANSWERED = true;

  showFeedback(LAST_OK, expected);
  inputEl.disabled = true;
```

```
  submitBtn.disabled = true;
  nextBtn.disabled = false;
  nextBtn.focus();
});

nextBtn.addEventListener("click", () => {
  if (!ANSWERED) return;
  goNext();
});

resetBtn.addEventListener("click", () => {
  clearCache(cacheKey(CONFIG.CARDS_PATH));
  startGame();
});

document.addEventListener("keydown", (e) => {
  if (e.key === "Enter" && ANSWERED && !nextBtn.disabled) {
    e.preventDefault();
    nextBtn.click();
  }
});

startGame();
```

☑ **What's removed:**

- *detectRepoContext*() and all dynamic repo logic.
- All references to *ctx* or auto-detected values.
- Replaced with direct use of CONFIG.OWNER, CONFIG.REPO, etc.

Let me know if you want to add a fallback for local testing or support for .json manifests instead of GitHub API calls. github.com github.com github.com

References

(1) GitHub - Fifteen Game
(2) GitHub - Webix/sources/models/progressch.js
(3) GitHub - OneKey Desktop

Conversation by Fifa Utuber with Copilot on Monday, Aug 11 2025